Advanced CI/CD with Jenkins on macOS + Docker Desktop

Student lab | Foundations and Efficient Operation of Modern Infrastructure | 2026/27/1

Mac-specific adaptation of the supplied WSL2 lab

Designed for a project folder already open in VS Code. Commands are intended for the integrated macOS terminal unless stated otherwise.

Key difference: Jenkins runs in Docker on the Mac. Docker builds inside Jenkins use a separate Docker-in-Docker service with TLS. No apt, WSL2, or systemd commands are run directly on macOS.

1. Learning outcomes and architecture

By the end, you will configure Jenkins in Docker on macOS, connect Jenkins to an isolated Docker-in-Docker (DinD) daemon, practice Git-based CI, run pytest tests, build a non-root application image, implement a declarative Jenkinsfile with a failure gate, verify HTTP endpoints, and diagnose deliberately broken builds.

This adapts the original Ubuntu WSL2 lab to your Mac. It uses Docker Desktop for Mac and keeps Jenkins itself in a container. It does not use apt on macOS, WSL2, systemd, or a native Linux Jenkins service.

```
macOS (Apple silicon / ARM64)
  +-- Docker Desktop
        +-- Jenkins container :8080
        |     +-- Git + Java 21 + Docker CLI
        |     +-- Jenkins jobs / plugins (persistent volume)
        +-- Docker-in-Docker service (TLS, isolated network)
              +-- inventory-candidate :5051 (temporary smoke test)
              +-- inventory-live      :5050 (local deployment)
  GitHub (main branch) --> Jenkins checkout --> tests --> image --> smoke --> deploy
```

Safety: keep Jenkins and the application local. Do not expose Jenkins to the public internet or commit credentials/tokens. Docker access is powerful; the DinD service below is privileged, so use it only for this isolated learning lab.

2. Prerequisites and preparation

You have already verified Docker with docker -v and docker run --rm hello-world. The output identified an ARM64 image, so this guide assumes Apple silicon unless your Mac reports otherwise.

In VS Code, keep your existing lab project folder open. Use Terminal → New Terminal. Confirm Docker Desktop is running, then run:

```
docker version
docker run --rm hello-world
docker info --format '{{.Architecture}}'
```

Expected architecture on Apple silicon is typically aarch64 or arm64, depending on the command/context. If Docker is working, do not install Linux package tools such as apt or try to create /etc/apt/keyrings on macOS.

Check the files already in your open folder:

```
pwd
ls -la
find . -maxdepth 2 -type f | sort
```

Look for the application Dockerfile, Jenkinsfile, app/, tests/, and requirements files. Do not replace your application Dockerfile with the Jenkins image Dockerfile; they serve different purposes.

3. Install Jenkins in Docker with Docker build capability

The official Jenkins image does not include the Docker CLI. This setup builds a small custom Jenkins image with Docker CLI and the Docker Pipeline plugins, then connects it to a separate DinD daemon using TLS.

Step A — create a folder named jenkins in the root of your existing project folder. Inside it, create a file named Dockerfile with this content:

```
FROM jenkins/jenkins:lts-jdk21

USER root

RUN apt-get update && apt-get install -y --no-install-recommends \\
    ca-certificates curl && \\
    install -m 0755 -d /etc/apt/keyrings && \\
    curl -fsSL https://download.docker.com/linux/debian/gpg \\
      -o /etc/apt/keyrings/docker.asc && \\
    chmod a+r /etc/apt/keyrings/docker.asc && \\
    echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \\
      https://download.docker.com/linux/debian \\
      $(. /etc/os-release && echo \\"$VERSION_CODENAME\\") stable" \\
      > /etc/apt/sources.list.d/docker.list && \\
```

```
    apt-get update && apt-get install -y --no-install-recommends docker-ce-cli && \\
    apt-get clean && rm -rf /var/lib/apt/lists/*

USER jenkins

RUN jenkins-plugin-cli --plugins "docker-workflow json-path-api"
```

Step B — create a file named docker-compose.jenkins.yml in the root of the project folder. This file runs Jenkins and its separate Docker daemon. It does not overwrite your existing app Dockerfile or application Compose file.

```
services:
  docker:
    image: docker:dind
    privileged: true
    restart: unless-stopped
    environment:
      DOCKER_TLS_CERTDIR: /certs
    networks:
      jenkins:
        aliases:
          - docker
    volumes:
      - jenkins-docker-certs:/certs/client
      - jenkins-docker-data:/var/lib/docker
    ports:
      - "127.0.0.1:2376:2376"

  jenkins:
    build:
      context: ./jenkins
    image: local/jenkins-docker:lts-jdk21
    restart: unless-stopped
    depends_on:
      - docker
    environment:
      DOCKER_HOST: tcp://docker:2376
      DOCKER_CERT_PATH: /certs/client
      DOCKER_TLS_VERIFY: "1"
    ports:
      - "127.0.0.1:8080:8080"
      - "127.0.0.1:50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - jenkins-docker-certs:/certs/client:ro
    networks:
      - jenkins

networks:
  jenkins:

volumes:
  jenkins_home:
  jenkins-docker-certs:
  jenkins-docker-data:
```

Step C — start the services from the project root (the folder containing docker-compose.jenkins.yml):

```
docker compose -f docker-compose.jenkins.yml up -d --build
docker compose -f docker-compose.jenkins.yml ps
docker compose -f docker-compose.jenkins.yml logs --tail=80 jenkins
```

The first build may take a few minutes. Jenkins is available only on your Mac's loopback interface at port 8080. If port 8080 is already occupied, change the left side of 127.0.0.1:8080:8080 to 127.0.0.1:8081:8080 and open http://localhost:8081.

4. Unlock and configure Jenkins

Get the initial administrator password:

```
docker compose -f docker-compose.jenkins.yml exec jenkins \\
  cat /var/jenkins_home/secrets/initialAdminPassword
```

Open http://localhost:8080. Paste the password, select Install suggested plugins, and create a named administrator account. In Manage Jenkins → Plugins, confirm Pipeline, Git, and JUnit are available.

Check Docker connectivity from the Jenkins container:

```
docker compose -f docker-compose.jenkins.yml exec jenkins sh -lc \\
  'docker version && docker info'
```

The client and server sections should both appear. If the client works but the server does not, inspect both services:

```
docker compose -f docker-compose.jenkins.yml logs --tail=100 docker
docker compose -f docker-compose.jenkins.yml logs --tail=100 jenkins
```

The DinD service is privileged, which grants substantial control over its own Docker environment. Keep this setup local and do not use it for shared university or production infrastructure.

5. Get the starter project and run checks locally

Use the starter project files supplied with your course. If they are already in the folder open in VS Code, do not extract another copy over them. Confirm the expected files exist:

```
ls -la
find app tests -maxdepth 2 -type f 2>/dev/null
ls Dockerfile Jenkinsfile requirements.txt requirements-dev.txt 2>/dev/null
```

If your starter files are in a subfolder, cd into that folder before running application commands. Run local tests on the Mac if Python is installed, or use the project's documented Python/container workflow:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
python -m pytest -v
```

Then build and smoke-test the app image. Adjust the image name and port mapping only if your supplied Dockerfile/README says otherwise:

```
docker build -t inventory-api:local .
docker run -d --name inventory-manual \\
  -p 127.0.0.1:5050:5000 inventory-api:local
curl -i http://127.0.0.1:5050/health
curl -i http://127.0.0.1:5050/api/items/999
docker rm -f inventory-manual
```

If requirements-dev.txt, app/, tests/, or the endpoints are absent, stop and inspect the actual starter files rather than inventing them. The original handout expects a Flask inventory API, but your local folder may differ.

6. Git and Jenkins pipeline-as-code

Create a GitHub repository for the starter project, for example inventory-ci-lab. Do not commit .venv, secrets, tokens, local Jenkins data, or credentials. Check .gitignore and .dockerignore before committing.

```
git status
git init -b main
git add .
git status
git commit -m "Initial inventory API with pipeline"
git remote add origin https://github.com/YOUR_USER/inventory-ci-lab.git
git push -u origin main
```

Replace YOUR_USER with your GitHub username. If the repository is already a Git repository, do not run git init again or add a second remote; inspect git status and git remote -v first.

In Jenkins, choose New Item → Pipeline. Name it InventoryPipeline. Under Pipeline, select Pipeline script from SCM, choose Git, enter the repository URL, set the branch to */main, and set Script Path to Jenkinsfile. Add a scoped credential only if required. Select Build Now.

The intended stages are Checkout → Unit tests → Build image → Container smoke test → Deploy locally. The smoke-test container should be removed even if a stage fails, and deploy should not run if tests fail. Review your supplied Jenkinsfile rather than assuming its contents match the handout.

7. Required engineering challenges

Challenge 1 — extend the API and tests (20 points). Add GET /api/items/low-stock?threshold=N. Return items with stock strictly below N. Validate N as an integer from 0 to 100 and return HTTP 400 with a JSON error for invalid values. Add at least five pytest cases covering boundaries, invalid input, and successful filtering.

Challenge 2 — prove the CI failure gate (15 points). Introduce a deliberate bug or failing test, commit it, and run Jenkins. Capture evidence that the test stage fails and image-build/deploy stages do not run. Fix it in a new commit and capture the successful run. Record both commit IDs.

Challenge 3 — improve deployment safety (20 points). Implement rollback to the previous working image/container or a documented blue/green traffic switch. Explain what happens to active requests during cutover and show failure-recovery evidence.

Challenge 4 — quality and security checks (15 points). Add a separate static-check stage, for example ruff check app tests. Use a non-root application user, exclude secrets and local virtual environments from the image build context, and ensure the final image does not contain development dependencies. Never put GitHub tokens in the Jenkinsfile.

Challenge 5 — design and reproducibility (10 points). Draw an architecture diagram and write a README with setup, endpoints, tests, pipeline stages, and error handling. Show that a fresh clone can run tests from declared requirements. Explain the risk of Docker socket/daemon access and a safer production alternative, such as an isolated ephemeral build agent.

Optional stretch challenges (up to 20 bonus points). Implement multibranch or pull-request builds that test/build but do not deploy, and/or add a dependency lock or image vulnerability scan with a published report.

8. Evidence to submit and marking scheme

Submit: (i) GitHub URL with source and Jenkinsfile; (ii) a 3–5 page engineering report covering environment versions, architecture, test output, Jenkins stages, failure-and-fix proof, smoke tests, security observations, and design decisions; (iii) at least four screenshots: successful pipeline, failed test stage, JUnit results, and live API response. Remove passwords and tokens from screenshots.

```
docker version
docker compose -f docker-compose.jenkins.yml ps
docker compose -f docker-compose.jenkins.yml logs --tail=80 jenkins
```

Baseline working environment (20 points); API and tests (20); failure gate (15); safer deployment (20); lint/security (15); documentation/reasoning (10). Total: 100 points, plus optional bonus.

9. Troubleshooting

Jenkins container will not start: run docker compose -f docker-compose.jenkins.yml logs --tail=100 jenkins. Check that Docker Desktop is running and that port 8080 is free.

Jenkins page does not open: run docker compose -f docker-compose.jenkins.yml ps; confirm the Jenkins service is Up and the port is published. Try http://localhost:8080. If you changed the host port, use that port.

Docker CLI works but builds fail: run the Docker connectivity check from section 4 and inspect DinD logs. The Jenkins service must reach docker:2376 over the Compose network with TLS certificates.

Build cannot find Dockerfile/Jenkinsfile: confirm the repository URL, branch */main, Script Path Jenkinsfile, and the file's location at the repository root. The application Dockerfile is not the Jenkins Dockerfile.

Smoke test fails: inspect docker logs inventory-candidate, the port mapping, container health, and /health response. Ensure the candidate port is not already in use.

Jenkins appears offline while installing plugins: inspect container logs and DNS/network connectivity. Avoid disabling TLS verification as a workaround.

Clean shutdown: docker compose -f docker-compose.jenkins.yml down. This stops/removes containers but preserves named volumes. To erase Jenkins configuration and build data as well, docker compose ... down -v deletes those volumes — do not run it unless you intend to reset everything.

10. References and adaptation note

Adapted from the supplied Advanced CI/CD with Jenkins on Ubuntu WSL2 student lab. The original environment uses native Jenkins in Ubuntu WSL2; this version uses Jenkins in Docker on macOS and a separate TLS-enabled Docker-in-Docker service.

Official references: Jenkins in Docker — https://www.jenkins.io/doc/book/installing/docker/ ; Jenkins download — https://www.jenkins.io/download/ ; Docker Desktop for Mac — https://docs.docker.com/desktop/setup/install/mac-install/ .

Version note: the guide uses the moving jenkins/jenkins:lts-jdk21 tag so the image can follow Jenkins LTS updates. For reproducible coursework, record the actual image digest/version used in your report.